

Machine Learning Distribuido

Semana 8

Agenda de la Sesión

01 Introducción a Machine Learning

02 Tipos de Aprendizaje

03 Conceptos Básicos de ML

04 Paradigmas de Paralelismo

05 Panorama de Apache Spark

01

Introducción a Machine Learning

Definición, historia y aplicaciones en el contexto de datos

¿Qué es Machine Learning?

Definición y relación con IA y Deep Learning

- **IA (Inteligencia Artificial):** campo amplio orientado a simular inteligencia humana
- **Machine Learning:** subconjunto de IA que permite a los sistemas **aprender de datos** sin ser programados explícitamente
- **Deep Learning:** subconjunto de ML basado en redes neuronales profundas

Historia y Evolución del ML

De la teoría a los modelos a gran escala

Época	Hito clave
1950s	Test de Turing — ¿pueden las máquinas pensar?
1960–80s	IA simbólica y sistemas expertos
1990s	Emergence del ML estadístico (SVM, Bayes)
2000s	Big Data y primeros clusters distribuidos
2010s	Revolución del Deep Learning (GPU, ImageNet)
2020s	Large Language Models, ML distribuido masivo

(Data Science Dojo, s.f.)

Aplicaciones del ML

- 🔍 **Optimización de consultas** — predicción de planes de ejecución más eficientes
- 🚨 **Detección de anomalías** — transacciones fraudulentas en tiempo real
- 🎯 **Sistemas de recomendación** — filtrado colaborativo sobre grandes datasets
- 🧠 **Mantenimiento predictivo** — fallo de nodos en clústeres
- 💬 **Búsqueda semántica** — embeddings vectoriales en bases de datos
- 📊 **ETL inteligente** — clasificación y limpieza automática de datos

02

Tipos de Aprendizaje

Supervisado · No Supervisado · Refuerzo · Semi-supervisado

Aprendizaje Supervisado

Aprender con datos etiquetados

- El modelo aprende la relación **entrada** → **salida** a partir de ejemplos etiquetados
- **Clasificación:** predice una categoría discreta
 - Ejemplos: detección de spam, diagnóstico médico
- **Regresión:** predice un valor continuo
 - Ejemplos: predicción de precios, demanda de energía

Algoritmos comunes:

- Regresión Logística / Lineal
- Support Vector Machine (SVM)
- Árboles de Decisión, Random Forest
- Redes Neuronales

(Sanfoundry, s.f.; Pandelu, s.f.)

Flujo:

Datos etiquetados → Entrenamiento →

Modelo → Predicción

Aprendizaje No Supervisado

Descubrir estructura en datos sin etiquetas

- No existen "respuestas correctas" — el modelo descubre patrones por sí solo
- **Clustering:** agrupar datos similares
 - K-Means, DBSCAN, Hierarchical Clustering
- **Reducción de dimensionalidad:** comprimir representaciones
 - PCA, t-SNE, UMAP, Autoencoders
- **Reglas de asociación:** encontrar ítems que co-ocurren
 - Apriori, FP-Growth (mercado de datos)

✓ Útil cuando no se dispone de datos etiquetados o el objetivo es **explorar** la estructura de los datos.

(DigitalOcean, s.f.)

Aprendizaje por Refuerzo

Aprender mediante interacción con el entorno

- Un **agente** toma acciones en un entorno y recibe **recompensas o penalizaciones**
- Objetivo: maximizar la recompensa acumulada a largo plazo
- No requiere datos etiquetados ni conjunto de entrenamiento fijo

Aplicaciones:

- 🎮 Juegos: AlphaGo, OpenAI Five
- 🤖 Robótica: manipulación de objetos
- 🚗 Vehículos autónomos
- 📈 Trading algorítmico

Semi-supervisado y Auto-supervisado

Paradigmas emergentes de gran impacto

Semi-supervisado

- Combina **pocos datos etiquetados** con grandes volúmenes sin etiquetar
- Reduce el coste de etiquetado manual
- Ej.: clasificación de imágenes médicas

Auto-supervisado

- Las **etiquetas se derivan del propio dato**
- El modelo aprende representaciones ricas
- Base de los grandes modelos actuales:
 - BERT (predice palabras enmascaradas)
 - GPT (predice el siguiente token)

03

Conceptos Básicos de ML

Features · Splits · Overfitting · Bias-Variance · Métricas · Regularización

Features y Representación de Datos

El vocabulario del modelo

- **Feature (característica):** variable de entrada que el modelo usa para aprender
- **Feature Engineering:** transformar datos brutos en representaciones útiles
 - Normalización / Estandarización
 - One-Hot Encoding (variables categóricas)
 - Embeddings (texto, imágenes)
- **Tipos de features:** numérico, categórico, ordinal, binario, texto, imagen, serie temporal

💡 La calidad de los features influye **más** en el rendimiento del modelo que la elección del algoritmo.

(MLU Explain, s.f.)

Conjuntos de Datos: Train / Val / Test

División típica:

Conjunto	Uso	% aprox.
Train	Ajustar parámetros	70–80%
Validation	Ajustar hiperparámetros	10–15%
Test	Evaluación final	10–15%

Técnicas avanzadas:

- **K-Fold Cross-Validation:** promedia k evaluaciones
- **Stratified Split:** preserva la distribución de clases
- **Time-Series Split:** respeta el orden temporal

⚠ *Data Leakage:* información del test que "contamina" el entrenamiento

Overfitting y Underfitting

Los dos enemigos del aprendizaje generalizable

Underfitting (Alto Bias)

- Modelo demasiado **simple**
- Error alto en train **y** en test
- Causas: pocas features, modelo lineal en problema no-lineal
- **Solución:** modelo más complejo, más features

Overfitting (Alta Varianza)

- Modelo demasiado **complejo**
- Error bajo en train, alto en test
- El modelo "memoriza el ruido"
- **Solución:** más datos, regularización, simplificar modelo, dropout

Bias-Variance Tradeoff

El equilibrio fundamental del ML

- **Bias:** error por suposiciones demasiado simples del modelo
- **Variance:** error por sensibilidad a fluctuaciones en los datos de entrenamiento
- **Ruido irreducible:** componente intrínseca de los datos

$$\text{Error Total} = \text{Bias}^2 + \text{Varianza} + \text{Ruido}$$

🎯 El objetivo es encontrar el punto de **complejidad óptima** donde el error total se minimiza — ni demasiado simple, ni demasiado complejo.

(Sanfoundry, s.f.; ACTE, 2026)

Métricas de Evaluación

¿Cómo saber si el modelo es bueno?

Clasificación:

- **Accuracy:** % de predicciones correctas
- **Precision:** $TP / (TP + FP)$
- **Recall:** $TP / (TP + FN)$
- **F1-Score:** media armónica de P y R
- **AUC-ROC:** área bajo la curva ROC

Regresión:

- **MAE:** error absoluto medio
- **MSE / RMSE:** error cuadrático medio
- **R²:** varianza explicada por el modelo

Herramienta clave:

- **Confusion Matrix:** visualiza TP, TN, FP, FN

Regularización

Controlar la complejidad para generalizar mejor

Técnica	Mecanismo	Efecto
L1 (Lasso)	Penaliza $\sum$	w_i
L2 (Ridge)	Penaliza $\sum w_i^2$	Reduce pesos pero no los anula
Elastic Net	Combina L1 + L2	Balance entre selección y shrinkage
Dropout	Desactiva neuronas aleatoriamente	Previene co-dependencias en redes neuronales
Early Stopping	Detiene el entrenamiento antes del sobreajuste	Usa la curva de validación

(Gorgo, s.f.)

04

Paradigmas de Paralelismo

Data Parallelism · Model Parallelism · Híbrido · Comunicación Distribuida

Paralelismo de Datos (Data Parallelism)

Escalar entrenamiento dividiendo el dataset

- El **dataset** se divide entre múltiples dispositivos/nodos
- Cada nodo mantiene una **copia completa del modelo**
- Los gradientes se sincronizan entre nodos al final de cada paso

Sincrónico:

- Todos los nodos sincronizan gradientes antes de actualizar
- Más consistente, más lento
-  Preferido para Deep Learning

(IBM, s.f.; Hu, s.f.)

Asincrónico:

- Cada nodo actualiza sin esperar a los demás
- Más rápido, posible inconsistencia
- Útil con nodos heterogéneos

Paralelismo de Modelos (Model Parallelism)

Distribuir el propio modelo entre dispositivos

- El **modelo** se divide entre múltiples GPUs/nodos
- Cada dispositivo computa una **porción del modelo**
- Necesario cuando el modelo **no cabe en una sola GPU**

Aplicaciones:

- GPT-4, LLaMA, modelos con cientos de billones de parámetros
- Modelos de visión de muy alta resolución

⚠ Requiere gestión cuidadosa del **flujo de activaciones** entre dispositivos — la comunicación entre capas puede ser el cuello de botella.

(Lei Mao, s.f.; NCBI, 2023)

Paralelismo Híbrido

Combinando estrategias para máxima escala

Tensor Parallelism

Divide operaciones matriciales individuales entre dispositivos (dentro de una capa)

Pipeline Parallelism

Distribuye las capas del modelo en etapas, procesando mini-batches en pipeline

Expert Parallelism (MoE)

Mixture of Experts: tokens se enrutan dinámicamente a distintos "expertos" en diferentes dispositivos

Enfoque moderno:

Combinar los tres para modelos de escala de billones de parámetros

(Towards AI, s.f.)

Comunicación Distribuida

El desafío de sincronizar gradientes a escala

Ring-AllReduce

- Nodos organizados en anillo
- Cada nodo envía y recibe gradientes en serie
- **Sin cuello de botella central**
- Usado por Horovod y NCCL

Parameter Server

- Servidor central almacena los parámetros
- Workers envían gradientes y reciben pesos actualizados
- Simple pero puede saturar el servidor

💡 El **ancho de banda de red** es frecuentemente el principal cuello de botella en entrenamiento distribuido.

(Seong, s.f.; Luo et al., 2024)

05

Panorama de Apache Spark

MLlib · Pipeline API · TorchDistributor

Spark MLlib

ML escalable integrado en Spark

- Librería de ML distribuida de Apache Spark — soporta **Java, Scala, Python, R**
- Procesa datasets que no caben en memoria de una sola máquina

Algoritmos disponibles:

Categoría	Ejemplos
Clasificación	Logistic Regression, Random Forest, GBT, Naive Bayes
Regresión	Linear Regression, Decision Tree Regression
Clustering	K-Means, Bisecting K-Means, LDA, GMM
Filtrado colaborativo	ALS (Alternating Least Squares)
Asociación	FP-Growth, PrefixSpan

(Apache Spark, s.f.; Developer Indian, s.f.)

Spark ML Pipeline API

Orquestar flujos de ML de forma reproducible

- **Transformer:** transforma un DataFrame en otro (`transform()`)
 - Ejemplos: `Tokenizer` , `StandardScaler` , `VectorAssembler`
- **Estimator:** ajusta un modelo a los datos (`fit()`) → devuelve un Transformer
 - Ejemplos: `LogisticRegression` , `KMeans` , `ALS`
- **Pipeline:** cadena de Transformers + Estimators → tratado como un Estimator

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.classification import RandomForestClassifier

assembler = VectorAssembler(inputCols=["f1","f2","f3"], outputCol="features")
scaler     = StandardScaler(inputCol="features", outputCol="scaled")
rf         = RandomForestClassifier(featuresCol="scaled", labelCol="label")
pipeline   = Pipeline(stages=[assembler, scaler, rf])
model      = pipeline.fit(train_df)
```

(Microsoft, s.f.)

TorchDistributor — Spark 4.0

PyTorch nativo sobre clústeres Spark

- Novedad de **Apache Spark 4.0** (mayo 2025)
- Permite ejecutar código **PyTorch directamente sobre Spark** con soporte GPU
- Las funciones de entrenamiento se ejecutan en múltiples Spark Executors simultáneamente
- Integración con la nueva arquitectura **Spark Connect** (clientes remotos)

```
from pyspark.ml.torch.distributor import TorchDistributor

def train_fn():
    import torch
    # ... código de entrenamiento PyTorch estándar
    return model

distributor = TorchDistributor(num_processes=4, local_mode=False, use_gpu=True)
result = distributor.run(train_fn)
```

(Databricks, 2025)

Caso de Uso End-to-End con Spark MLlib

Sistema de recomendación con ALS

```
from pyspark.sql import SparkSession
from pyspark.ml.recommendation import ALS
from pyspark.ml.evaluation import RegressionEvaluator

spark = SparkSession.builder.appName("Recomendador").getOrCreate()

# 1. Cargar datos
ratings = spark.read.parquet("s3://bucket/ratings/")

# 2. Dividir dataset
train, test = ratings.randomSplit([0.8, 0.2], seed=42)

# 3. Entrenar modelo ALS
als = ALS(maxIter=10, regParam=0.1, userCol="userId",
          itemCol="movieId", ratingCol="rating", coldStartStrategy="drop")
```

¿Preguntas?

Referencias

ACTE. (2026). *The bias and variance tradeoff: Balancing accuracy.*

<https://www.acte.in/understanding-bias-variance-tradeoff>

AlmaBetter. (s.f.). *Setting up a distributed ML environment with Apache Spark.*

<https://www.almabetter.com/bytes/tutorials/mlops/set-up-distributed-ml-environment-with-apache-spark>

Apache Spark. (s.f.). *Machine learning library (MLlib) guide.*

<https://spark.apache.org/docs/latest/ml-guide.html>

Apache Spark. (s.f.). *MLlib.* <https://spark.apache.org/mlib/>

Certometer. (s.f.). *Understanding bias, variance, overfitting, underfitting, and the tradeoff.*

<https://www.certometer.com/blogs/machine-learning/understanding-bias-variance-overfitting-underfitting-and-the-tradeoff>

Data Science Dojo. (s.f.). *Machine learning 101: The types of ML explained.*

<https://datasciencedojo.com/blog/machine-learning-101/>

DataCamp. (s.f.). *Bias-variance tradeoff: How models fail in production.*

<https://www.datacamp.com/tutorial/bias-variance-tradeoff>

Databricks. (2025). *Apache Spark 4.0: The biggest leap for machine learning since Spark 2.0.*

<https://community.databricks.com/t5/technical-blog/apache-spark-4-0-a-new-era-for-scalable-machine-learning-and-ai/ba-p/120627>

DEV Community. (s.f.). *Mastering distributed machine learning: How to 10X your PyTorch training speed with Ray & DDP*. <https://dev.to/m-a-h-b-u-b/mastering-distributed-machine-learning-how-to-10x-your-pytorch-training-speed-with-ray-ddp-5hgg>

Developer Indian. (s.f.). *Apache Spark MLlib: A complete guide to scalable machine learning*. <https://www.developerindian.com/articles/apache-spark-mllib-a-complete-guide-to-scalable-machine-learning>

Digital Regenesys. (s.f.). *Types of machine learning in AI – supervised, unsupervised & more*. <https://www.digitalregenesys.com/blog/types-of-machine-learning-in-artificial-intelligence>

DigitalOcean. (s.f.). *Types of machine learning: Supervised, unsupervised, and more*. <https://www.digitalocean.com/resources/articles/types-of-machine-learning>

GeeksforGeeks. (s.f.). *Introduction to MLlib – Apache Spark*.

<https://www.geeksforgeeks.org/data-science/introduction-to-mlib-apache-spark/>

Gorgo, L. (s.f.). *Bias-variance tradeoff in machine learning*. Medium.

<https://leonidasgorgo.medium.com/bias-variance-tradeoff-in-machine-learning-34d62b17af9a>

Horovod Project. (s.f.). *Horovod*. <https://horovod.ai/>

Hu, L. (s.f.). *Distributed parallel training: Data parallelism and model parallelism*. Towards Data Science. <https://towardsdatascience.com/distributed-parallel-training-data-parallelism-and-model-parallelism-ec2d234e3214/>

IBM. (s.f.). *What is distributed machine learning?* <https://www.ibm.com/think/topics/distributed-machine-learning>

Introl. (2025). *Ray clusters for AI: Distributed computing architecture.*
<https://introl.com/blog/ray-clusters-distributed-ai-computing-infrastructure-guide-2025>

Lei Mao. (s.f.). *Data parallelism vs model parallelism in distributed deep learning training.*
<https://leimao.github.io/blog/Data-Parallelism-vs-Model-Parallelism/>

Luo, Z., et al. (2024). *Cyclic data parallelism for efficient parallelism of deep neural networks.*
arXiv. <https://arxiv.org/pdf/2403.08837>

Malik, S. (s.f.). *Types of machine learning: Supervised, unsupervised, and reinforcement explained*. Medium. <https://medium.com/@sohaibmalikdev/>

Meng, X., et al. (2016). *MLlib: Machine learning in Apache Spark*. *Journal of Machine Learning Research*, 17(1), 1–7. <https://www.jmlr.org/papers/volume17/15-237/15-237.pdf>

Microsoft. (s.f.). *Train machine learning models with Apache Spark – Microsoft Fabric*. <https://learn.microsoft.com/en-us/fabric/data-science/model-training-overview>

Pandelu, A. (s.f.). *Day 1: What is machine learning?* Medium. <https://medium.com/@bhatadithya54764118/day-1-what-is-machine-learning-3060d8121d11>

Pandey, S. (s.f.). *Apache Spark for machine learning*. Medium. <https://medium.com/@thisis-Shitanshu/apache-spark-for-machine-learning-fdef4dcbe1d7>

Pecan AI. (s.f.). *3 types of machine learning in 2026*. <https://www.pecan.ai/blog/3-types-of-machine-learning/>

Ray Project. (s.f.). *Get started with distributed training using Horovod*. Ray Docs. <https://docs.ray.io/en/latest/train/horovod.html>

Sanfoundry. (s.f.). *Bias-variance tradeoff in machine learning*. <https://www.sanfoundry.com/bias-variance-tradeoff-in-machine-learning/>

Sergeev, A., & Del Balso, M. (2018). *Horovod: Fast and easy distributed deep learning in TensorFlow*. arXiv. <https://arxiv.org/pdf/1802.05799>

Seong, S. (s.f.). *Data parallelism in machine learning training*. Medium.

<https://medium.com/cloudvillains/data-parallelism-in-machine-learning-training-686ed9ab05fb>

TechTarget. (s.f.). *4 types of machine learning models explained*.

<https://www.techtarget.com/searchenterpriseai/tip/Types-of-learning-in-machine-learning-explained>

Towards AI. (s.f.). *Machine learning at scale: Model v/s data parallelism*.

<https://towardsai.net/p//machine-learning-at-scale-model-v-s-data-parallelism>

Uplatz. (s.f.). *Architectures for scale: A comparative analysis of Horovod, Ray, and PyTorch Lightning for distributed deep learning*. <https://uplatz.com/blog/architectures-for-scale-a-comparative-analysis-of-horovod-ray-and-pytorch-lightning-for-distributed-deep-learning/>